

Python Automation — Module 3

Sous-module : APIs Data Source

Fil rouge : Dataset Transilien — SNCF Open Data

1. Contexte et objectif

Ce sous-module s'inscrit dans la démarche de diversification des sources de données. L'objectif est de remplacer le téléchargement manuel d'un fichier CSV par une récupération dynamique via API — base de tout pipeline de données réel.

Le dataset choisi comme fil rouge est le comptage des voyageurs montants dans les trains Transilien, publié par SNCF sur ressources.data.sncf.com.

1.1 Présentation du dataset Transilien

Chaque enregistrement représente un comptage de voyageurs montants dans un train Transilien, pour :

- une gare donnée
- une ligne donnée (B, C, H, J, L, N, P, R, U...)
- un type de jour : JOB (jour ouvrable), SAM (samedi), DIM (dimanche)
- une tranche horaire donnée

Les comptages sont réalisés soit automatiquement (capteurs embarqués), soit manuellement (environ une fois tous les 4 ans). Il n'y a pas d'historisation : chaque mise à jour remplace les données précédentes.

Source : SNCF Open Data — plateforme OpenDataSoft — sans clé API requise.

1.2 Ce que change l'API par rapport au CSV

En passant du CSV téléchargé à l'API, on passe d'une source statique et figée à une récupération dynamique, paramétrable et intégrable dans un pipeline automatisé. Le script ne dépend plus d'un fichier local : il interroge directement la source à la demande.

2. Mécanique d'une API REST

Une API REST est une adresse web à laquelle on pose une question. Cette adresse s'appelle un endpoint. On construit la question en ajoutant des paramètres à cette adresse.

2.1 Structure d'une URL d'API

Une URL d'API se décompose en trois parties :

```
https://ressources.data.sncf.com/api/explore/v2.1/catalog/datasets/  
comptage-voyageurs-trains-transilien/records  
?limit=10&offset=0&where=ligne="H"
```

- La base : l'adresse du serveur et du dataset — fixe, ne change jamais
- L'endpoint : /records — la ressource demandée (des enregistrements)
- Les paramètres : tout ce qui suit le ? — la question affinée

2.2 Les paramètres principaux

Paramètre	Rôle	Exemple
limit	Nombre d'enregistrements par réponse (max 100)	limit=100
offset	Point de départ dans le dataset — pilote la pagination	offset=100
where	Filtre sur les données — équivalent SQL WHERE	where=ligne="H"
refine	Filtre rapide sur une valeur exacte	refine=type_jour:JOB

2.3 Pourquoi une réponse JSON ?

Le JSON est le format universel d'échange sur le web. Il est lisible par tous les langages (Python, JavaScript, R...). La réponse contient deux clés principales :

- total_count : nombre total d'enregistrements correspondant à la requête — pilote la pagination
- results : liste des enregistrements de la page courante

3. Exploration dans le notebook

Le notebook `transilien_api_exploration.ipynb` sert de terrain d'exploration — comprendre la mécanique avant d'encapsuler dans un script.

3.1 Premier appel brut

```
import requests

url =
"https://ressources.data.sncf.com/api/explore/v2.1/catalog/datasets/
    comptage-voyageurs-trains-transilien/records"
params = {"limit": 5}

response = requests.get(url, params=params)
print(response.status_code) # 200 = succès
print(response.json())
```

Le status code 200 confirme que la requête a abouti. La réponse révèle 7328 enregistrements au total dans le dataset.

3.2 Affichage lisible du JSON

```
import json
print(json.dumps(response.json(), indent=2, ensure_ascii=False))
```

indent=2 indente chaque niveau pour la lisibilité. ensure_ascii=False préserve les caractères français (accents, tirets).

3.3 Filtrage avec where

```
params = {
    "limit": 5,
    "where": 'ligne="H"'
}
```

La condition ligne="H" est une expression texte envoyée à l'API — pas du Python. Avec ce filtre, total_count passe de 7328 à 200 : uniquement les enregistrements de la ligne H.

3.4 Pagination avec while

L'API retourne 100 enregistrements maximum par appel. Pour récupérer l'ensemble, on boucle en incrémentant offset de 100 à chaque tour.

```
total_count = 200
offset = 0
results = []
```

```

while offset < total_count:
    url = "https://..."
    params = {"limit": 100, "where": 'ligne="H"', "offset": offset}
    response = requests.get(url, params=params)
    results.extend(response.json()["results"])
    offset += 100

```

`results.extend()` aplatit directement chaque page dans la liste principale, au lieu d'une liste de listes.

3.5 Conversion en DataFrame pandas

```

import pandas as pd

df = pd.DataFrame(results)
df.shape # (200, 9)

```

`results` est déjà une liste de dictionnaires — le format natif que pandas attend. Chaque dictionnaire devient une ligne, chaque clé une colonne.

4. Le script réutilisable — `fetch_transilien.py`

Une fois la mécanique comprise dans le notebook, le code est encapsulé dans un script Python pur, dans VS Code. Ce script n'est pas un exercice : c'est une fonction d'ingestion réutilisable, callable depuis un pipeline ou un orchestrateur (Prefect — Mois 5).

4.1 Code complet

```

import requests
import pandas as pd

def fetch_transilien(ligne_train=None):
    url = "https://ressources.data.sncf.com/api/explore/v2.1/catalog/datasets/comptage-voyageurs-trains-transilien/records"

    params = {"limit": 100, "offset": 0}

    if ligne_train:
        params["where"] = f'ligne="{ligne_train}"'

```

```

# Premier appel : récupère total_count + première page
response = requests.get(url, params=params)
data = response.json()
total_count = data["total_count"]
results = data["results"]

# Pagination
while params["offset"] + 100 < total_count:
    params["offset"] += 100
    response = requests.get(url, params=params)
    results.extend(response.json()["results"])

return pd.DataFrame(results)

```

4.2 Structure par blocs

Le script est organisé en 6 blocs distincts :

Bloc	Rôle
Import	requests pour les appels HTTP, pandas pour le DataFrame
Définition de la fonction	Un paramètre optionnel ligne_train=None — sans filtre par défaut
Construction des params	Paramètres de base fixes + filtre where ajouté conditionnellement
Premier appel	Récupère total_count dynamiquement + première page de résultats
Pagination	Boucle while — avance de 100 en 100 jusqu'à exhaustion
Retour	pd.DataFrame(results) — une ligne, un livrable propre

4.3 Usage

```

# Tout le dataset
df = fetch_transilien()

# Une ligne précise
df = fetch_transilien(ligne_train="H")

# Depuis le terminal VS Code
python -c "from fetch_transilien import fetch_transilien;
print(fetch_transilien(ligne_train='H').shape)"

```

Le paramètre `ligne_train` représente ce qui varie selon le besoin analytique — pas une ligne du `DataFrame`. Le nom évite toute confusion avec les lignes pandas.

5. Bilan

5.1 Ce que ce sous-module a couvert

- La mécanique fondamentale : requête HTTP → JSON → `DataFrame` pandas
- Les paramètres `limit`, `offset`, `where` et leur rôle respectif
- La pagination : pourquoi elle est nécessaire et comment l'automatiser
- La différence entre notebook d'exploration et script réutilisable
- L'environnement d'exécution : VS Code pour les scripts, Jupyter pour l'analyse

5.2 Ce qui a été difficile

La partie la plus abstraite est la pagination : on ne parcourt pas une liste visible, on construit une boucle qui avance dans un dataset distant, page par page, sans jamais le voir en entier. Passer par cette abstraction est nécessaire — c'est ce qui permet de remplacer un téléchargement manuel figé par une récupération dynamique intégrable dans un pipeline.

5.3 Connexion avec la suite

- APIs LLM (Bloc 2) : même mécanique HTTP, destination différente — génération d'insights textuels
- Mois 5 — Prefect : `fetch_transilien()` sera callable depuis un orchestrateur, sans intervention manuelle

6. Structure des fichiers

```
module3-apis/  
├── APIs LLM/  
└── APIs data source/  
    ├── transilien_api_exploration.ipynb  
    └── fetch_transilien.py
```